0 8 8

T fixtures — 0 3 `sibling_search_check.sh` RED-test corpus (plan.md T-D)

Revision: 1 **Last modified:** 2026-09-29T11:02:58+05:00

Contract: `specs/004-fast-dev-cycles/contracts/closure-refusal.md` CR-004 (sibling search) + `data-model.md` §6.4 `SiblingSearchArtefact` + `common-conventions.md` C-001 (exit codes) / C-004 (evidence and needles) / C-005 (self-validation triple). tasks.md T088, verbatim: "RED tests `constitution/scripts/fastcycle/tests/test_sibling_search_check_red.sh` and `constitution/scripts/workable-items/cmd/workable-items/fastcycle_sibling_search_test.go` (Bug closure without the artefact accepted today; golden-bad: refused; golden-bad: an artefact whose search shows no control needle is rejected; negative control: a Task closes without it) (plan T-D03; FR-011, SC-004)". plan.md T-D03 "Protecting tests" line, verbatim: "RED: closure without the artefact accepted today; golden-bad: refused; golden-bad: an artefact whose search shows no needle is rejected (a null without a needle is not evidence); negative control: a Task closes without it; paired mutation: skip the needle check → the second golden-bad FAILs." 0 9 8

The tool under test (not yet implemented — T , a later task)

`constitution/scripts/fastcycle/closure/sibling_search_check.sh`, per plan.md's path table (line 199: `sibling_search_check.sh # FR-011 artefact validator (T-D03)`). Per T096's own "Work" line ("Implement `constitution/scripts/fastcycle/closure/sibling_search_check.sh` (validates class statement, search method with its control needle, every instance with disposition

fixed-here / tracked-as ATM-nnn / proven-not-an-instance) until the shell half of T088 is GREEN"), the real implementation reads one `SiblingSearchArtefact` JSON file (data-model.md §6.4) and validates it.

Inherits `common-conventions.md` in full, in particular:

- **C-001** exit codes: `0` the artefact is VALID (an ACCEPTED `SiblingSearchArtefact`); `1` a FINDING — the artefact is present but INVALID, naming the offending field/record on stdout; `2` usage error (missing/unreadable positional argument); `3` self-test failure; `4` BLIND (the artefact file could not be read at all — distinct from "read fine but invalid", per C-001's own BLIND row).
- **C-004** control needles: `instances_found` being empty is NEVER, by itself, evidence that no sibling instance exists — it is evidence ONLY when `control_needle_found` is independently `true`, proving the SAME search method that produced the empty list can genuinely find a known-present thing (§11.4.201(7)(b) / §11.4.273(f): "a null without a needle is not evidence", plan.md T-D03's own wording, verbatim above).
- **C-005** self-validation triple, using the shared `constitution/scripts/fastcycle/tests/lib/triple_harness.sh` runner (already implemented; this RED test invokes it directly, so it self-flips from "bad --tool" (exit 2, tool absent) to real per-fixture verdicts the moment T096 lands — no edit to this RED test needed).

Assumed CLI contract (UNCONFIRMED by the plan/contract text itself — no prior artefact names an exact argv shape; DEFINED here for this RED test's own GREEN-branch, binding-if-adopted on T096's implementer, following the house precedent in `fixtures/io_trace/README.md` and `fixtures/verdict_cache/README.md`):

```
sibling_search_check.sh <artefact.json>
```

Reads the single positional `<artefact.json>` path and validates it against the `SiblingSearchArtefact` schema below. On a finding (exit 1), prints at least one line naming the offending field/record — `triple_harness.sh`'s own C-005 "whole token" name-matching rule governs which literal words each `golden-bad*/expected` file below requires on stdout.

6 4

`SiblingSearchArtefact` JSON schema (data-model.md § . , this RED test's

own concretisation of the abstract field list into JSON)

```
{
  "class_statement": "<one-line statement of the defect class being searched for>",
  "search_method": "<the command/query used to search for sibling instances>",
  "control_needle": "<a known-present instance the search_method MUST find>",
  "control_needle_found": true,
  "instances_found": [
    {"location": "<path:line>", "disposition": "fixed-here"},
    {"location": "<path:line>", "disposition": "tracked-as ATM-953"},
    {"location": "<path:line>", "disposition": "proven-not-an-instance", "proof": "<why>"}
  ]
}
```

`disposition` is drawn from the DEC-20 closed set (data-model.md §6.4): `fixed-here` / `tracked-as <ItemId>` (an `[A-Z]{2,}-[0-9]+` ticket id, matching this project's own `canonicalIDRe` , `constitution/scripts/workable-items/cmd/workable-items/crud.go:25`) / `proven-not-an-instance` . `ContentAddress` (data-model.md §6.4's final field) is treated here as tool-computed provenance metadata, out of this RED test's scope — no fixture below sets it, and none of the golden-bad classes exercises its absence.

The `scenarios`

Directory	Scenario	Protecting-tests line it satisfies
`golden-good/`	**Golden-good**	a well-formed artefact: needle found, every instance has a valid disposition
`golden-bad/`	**Golden-bad (required by C-005)**	"golden-bad: refused" — `search_method` is missing entirely
`golden-bad-needle-not-found/`	**Golden-bad (2nd class, T-D03's own 2nd bullet)**	"golden-bad: an artefact whose search shows no control needle is rejected"
`golden-bad-invalid-disposition/`	**Golden-bad (3rd class)**	T096's own scope line ("every instance with disposition fixed-here / tracked-as ATM-nnn / proven-not-an-instance") — a disposition outside that closed set
`negative-control/`	**Negative control (§11.4.201(1) false-positive guard)**	zero `instances_found`, BUT `control_needle_found: true` — a genuinely-empty result the needle proves trustworthy, per plan.md T-D03's own "a null WITHOUT a needle is not evidence" (implying a null WITH one IS)

`golden-good/input` — golden-good

`control_needle_found: true` and every one of 3 `instances_found` entries carries a disposition from the closed set (one of each: `fixed-here`, `tracked-as ATM-953`, `proven-not-an-instance`). `sibling_search_check.sh golden-good/input` must exit 0.

`golden-bad/input` — golden-bad (required)

Identical to `golden-good/input` except the `search_method` key is entirely absent. `sibling_search_check.sh golden-bad/input` must exit 1 and its stdout must contain the whole token `search_method` (this fixture's `expected` file).[2]

`golden-bad-needle-not-found/input` — golden-bad (nd class)

Identical to `golden-good/input` except `control_needle_found: false` (the search ran, but the mechanism's own proof-of-life needle was NOT found — the mechanism is not trustworthy, so its result, including any `instances_found` entries, cannot be trusted either). `expected` requires the whole token `control_needle`.[3]

`golden-bad-invalid-disposition/input` — golden-bad (rd class)

Identical to `golden-good/input` except one `instances_found` entry's `disposition` is the literal string `"under-investigation"` — a value outside the DEC-20 closed set `{fixed-here, tracked-as <ItemId>, proven-not-an-instance}`. `expected` requires the whole token `disposition`.

`negative-control/input` — negative control

`control_needle_found: true`, `instances_found: []` (a genuinely empty list). This is the §11.4.201(1) false-positive guard for `golden-bad-needle-not-found/`: a checker that rejects EVERY empty-or-small `instances_found` list, rather than specifically the ones whose needle FAILED, would wrongly reject this fixture too. `sibling_search_check.sh negative-control/input` must exit 0 — zero sibling instances is a legitimate, evidence-backed finding when the search mechanism's own control needle proves it looked and could see.